Predicting in latent space

A technical proposal for an alternative simulation-to-world-model pipeline that predicts representations instead of events, and trains a vertical world model on the paired censored and uncensored views only a simulation can provide.

Fredrik Paulin · June 2026 · technical whitepaper / proposal draft

---

Abstract

This is a second route to the same destination as *From simulation to a vertical world model*. That proposal trains a generative world model that predicts the next event in a tokenized stream. This one trains a **joint-embedding predictive architecture** (JEPA): instead of reconstructing the next event, it predicts the *representation* of the future from the representation of the censored present, and discards the unpredictable surface detail a generative model is obliged to model. The argument for doing so is sharper in this domain than in the vision domain JEPA was built for, because the nuisance variable can be named — which individual customer arrives when — while the predictable structure (the demand regime, the stock trajectory, the cash position) is exactly what a manager needs. The simulation supplies an unusually clean training signal for this: for any world-state it can emit both the censored manager-view and the uncensored truth-or-future view, so a joint-embedding model can be trained to encode, from the censored present, the latent that predicts the truth. De-censoring becomes representation learning rather than explicit estimation. We keep the generator and corpus of the companion pipeline, add paired views, replace the generative training objective with a non-contrastive joint-embedding objective, and make the downstream artifact an action-conditioned latent model that plans by rolling actions forward in representation space. We give the architecture, the algorithms, an evaluation protocol, an honest comparison against the generative pipeline, and the failure modes — chief among them representation collapse, which this approach must actively prevent and the generative one never risks.

Companion papers

This is the third document in a set. The position paper — *Manufacture the data, generalize the model, ground the agent* (`position-paper-simulated-world-models.md`) — argues why a mechanistic simulation of a data-poor industry can supply the data and a world model can generalize across it. The first pipeline paper — *From simulation to a vertical world model* (`whitepaper-sim-to-worldmodel-pipeline.md`) — specifies a **generative** route: tokenize events, predict the next one. This document specifies a **joint-embedding** route: predict in latent space. The two pipelines share Stage 1 (the generator) and most of Stage 2 (the corpus), and diverge at training and use. Section 9 is an honest guide to which to prefer. Read the position paper for motivation, the generative paper for the default route, and this one for the alternative.

1. Scope and what changes

The four-stage pipeline is unchanged in outline — generate, aggregate, train, adapt — but two stages differ from the generative proposal. The table states the delta precisely.

Stage	Generative pipeline	This (joint-embedding) pipeline
1. Generate	Mechanistic, θ-randomized simulation	Same generator, **plus paired-view emission** (§4)
2. Aggregate	Event trajectories + truth + outcome	Same, **plus aligned (context, target) pairs** (§4)
3. Train	Autoregressive next-event likelihood	**Non-contrastive joint-embedding prediction in latent space** (§5–6)
4. Adapt / use	Autoregressive rollout; generative heads	**Latent-space planning (MPC) + probe heads** (§7)

What is reused verbatim: the determinism, mechanistic grounding, conservation constraints, θ-randomization, schema parity, and uncensored labelling of the generator and corpus. The reader is referred to the generative paper for those; this document specifies only what changes, plus the parts of training and use that are wholly new.

The worked vertical remains a perishable-goods retailer (a bakery), used only as a concrete archetype.

2. Motivation: predict the structure, discard the noise

A generative world model trained on next-event likelihood must model everything in the event stream, including the parts that are pure noise. In a perishable-goods business the dominant surface noise is identity and timing at the individual level: which customer arrives, in which second, for which of two near-substitute items. None of it is predictable and none of it matters to a decision. What matters — the demand regime, how stock will decay, how a missed delivery propagates, where cash lands — is lower-dimensional, slower, and predictable. A model rewarded for next-token accuracy spends capacity on the noise to lower its loss; this is the same objection LeCun (2022) raises against pixel-generative world models, and the motivation for predicting in an abstract representation space instead.

Joint-embedding predictive architectures act on that objection. I-JEPA (Assran et al., 2023) predicts the representations of masked image regions from a context region rather than their pixels; V-JEPA (Bardes et al., 2024) does the same for video in feature space; the principle is that the predictor is forced to synthesize semantic structure because it is denied the low-level detail. The bet of this paper is that it pays off more cleanly here than in vision, because in a business the nuisance is not merely high-dimensional, it is *nameable* — and a generator that knows which detail is noise can withhold exactly that detail from the target representation.

There is a second, stronger reason, specific to this problem. The quantity the business most needs is latent by construction: true demand is the hidden cause, censored sales are its observable shadow, and recovering one from the other is a hard estimation problem with its own literature (FreshRetailNet-50K, 2025). A model whose entire job is to predict a latent representation — one it never has to decode back to surface counts — is a natural model of

“the state behind the till.” That alignment between what JEPA learns and what the business needs is the reason this is worth a whole pipeline rather than a footnote.

3. The paired-view principle

A joint-embedding model is trained on pairs: a context, a target, and an asymmetry that prevents the trivial solution of copying. In vision the pair is two regions of one image and the asymmetry is masking. Here the simulation provides a pairing no real dataset can: for a single ground-truth world-state it emits two views.

- **Context view** — what a manager sees: the censored, lagged, sales-only projection. This is the observation contract the production harness also obeys.
- **Target view** — what reality hides: the uncensored truth (true demand including lost sales, full cost basis) and/or the future state a horizon ahead.

Training a joint-embedding model to predict the target representation from the context representation forces the context embedding to encode the latent that explains the truth behind the censored present. This is privileged-information training — the target encoder may see what the context encoder may not — and it turns de-censoring from an explicit estimator into a property of the learned representation. No quantity of real data can supply this pairing, because reality never records the target view. It is the single largest reason to prefer this route, and it is only available because the data is simulated.

4. Generator and corpus: the additions

The generator and corpus are those of the companion pipeline, with one addition at each stage.

Generation (addition). At each emitted step the generator records, alongside the observable event, the aligned ground-truth and a future snapshot at one or more horizons. Because the simulation holds the full state (it produced both demand and sale), these views are exact, not estimated.

Corpus (addition). The training unit gains an explicit pairing. A record is a trajectory of (context, target) pairs plus the action taken and the realized outcome:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "type": "object",
  "required": ["theta", "seed", "pairs", "outcome"],
  "properties": {
    "theta": { "type": "object", "description": "industry coordinate (θ)" },
    "seed":  { "type": "integer" },
    "pairs": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["context", "target"],
        "properties": {
          "context": { "type": "object", "description": "censored, lagged manager-view at t" },
          "action":  { "type": "object", "description": "decision taken at t, if any" },
          "target":  { "type": "object", "description": "uncensored truth / future at t+h" },
          "horizon": { "type": "integer", "description": "steps ahead the target looks" }
        }
      }
    },
    "outcome": { "type": "object", "description": "realized return: profit, service, waste" }
  }
}
```

Money remains integer minor units; the context view carries both `occurredAt` and `receivedAt` so the encoder learns to represent staleness. Encoders for the heterogeneous structured views are needed (§6.1); the corpus stores the raw views, not embeddings, so the encoder can be retrained without regenerating data.

5. The training objective

The model has three learned parts: a **context encoder** f, a **target encoder** g, and a **predictor** h. Given a context view x and a target view y, the loss minimizes the distance between the predicted and actual target *representations*:

```
L_pred = D( h(f(x), a, horizon),  sg[ g(y) ] )
```

where a is the action (when the step is a decision), `sg` is stop-gradient, and D is a regression distance (smooth-L1 or cosine). Prediction is in representation space; there is no reconstruction term and no pixel- or token-level decoder in the objective.

The objective has one well-known hazard: **representation collapse**, where f and g map everything to a constant and the loss is trivially zero. The generative pipeline never risks this; this one must prevent it explicitly, by one of two established routes:

1. **Asymmetric target with EMA.** g is not trained by gradient; it is an exponential moving average of f, the mechanism BYOL (Grill et al., 2020) and I-JEPA (Assran et al., 2023) use. Combined with the predictor and stop-gradient, the asymmetry blocks collapse.
2. **Variance-covariance regularization.** Add VICReg terms (Bardes et al., 2022) — a hinge on per-dimension embedding variance and an off-diagonal covariance penalty — to keep the representation spread out and decorrelated.

A robust default, following DINO-WM (Zhou et al., 2024), is to predict in the latent space of a **frozen** encoder: pretrain (or adopt) an encoder of the business state, freeze it, and learn only the predictor on top. DINO-WM reports that a JEPA-style model over frozen DINOv2 features outperforms generative latent world models (DreamerV3, TD-MPC2) on goal-conditioned planning, while sidestepping collapse entirely because the encoder is never trained end-to-end. For a small team this is the lowest-risk starting point.

6. The world model

6.1 Encoders for structured business state

Unlike images, the views here are heterogeneous structured records (typed events, lot sets, ledger balances). The encoder is a set/sequence encoder over the view's fields — a small transformer over a factored field embedding, with numeric channels scaled and embedded rather than quantized (the latent target tolerates approximate numerics in a way a generative head does not). The output is a fixed-width state embedding. The same encoder architecture serves context and target; only their inputs (censored vs uncensored) differ.

6.2 Hierarchy: predicting at the business's natural cadences

A bakery has dynamics at several time scales — intraday demand, weekly rota and ordering, seasonal drift. LeCun's hierarchical JEPA (H-JEPA) predicts at multiple temporal abstractions; that maps directly onto the cadence axis the harness already uses. We propose a two- or three-level H-JEPA: a fast level predicting next-hours latents, a slow level predicting next-days/weeks latents conditioned on the fast level's summary. The slow level is where

seasonal and procurement structure lives; the fast level is where intraday demand shape lives.

6.3 Action conditioning

To inform decisions the predictor must be action-conditioned: h takes the context latent *and* a candidate action and predicts the resulting target latent. This is the V-JEPA 2-AC construction (V-JEPA 2, 2025): a frame-causal, action-conditioned predictor on top of a frozen video encoder, used inside a planning loop. The same structure lets the harness ask "if I bake 20% more / reorder now / move Sara to mornings, what latent state results, and what does it cost?"

6.4 Probe heads: turning latents into decisions

A representation is not yet an order quantity. Lightweight heads are probed off the context (and predicted) latents:

- a **demand head** emitting a calibrated distribution (quantiles / CRPS-scored), de-censored by §3;
- a **value head** estimating the realized outcome (operating profit, service level) for the action-conditioned rollout;
- optional **decoders** for human-readable quantities the cockpit displays.

The heads are small and trained by supervised regression against the corpus labels, on top of frozen or slowly-updated representations. The world model is the encoder-plus-predictor; the heads are how the harness reads it.

7. Algorithms

Notation as in the companion pipeline: Θ the parameter support, H the horizon, f/g/h the context encoder / target encoder / predictor.

Algorithm 1 — Paired-view corpus generation.

```
for i in 1..N:
    θ ← sample(θ; space-filling); s ← seed(); π ← sample(behaviour_portfolio)
    state ← init(θ, s)
    for t in 1..H:
        ctx   ← censored_view(state)                # manager-view: lagged, sales-only
        a     ← π(ctx)
        state ← step(state, a, rng(s,t))            # mechanistic, conserved
        tgt   ← truth_view(state, horizon)          # uncensored + future (exact)
        emit_pair(θ, s, ctx, a, tgt, horizon)
    emit_outcome(score(truth_trajectory))
```

Algorithm 2 — Joint-embedding pretraining (collapse-safe).

```
for batch of (ctx, a, tgt) pairs:
    zc ← f(ctx)                                     # context embedding (trained)
    zt ← sg[ g(tgt) ]                               # target embedding (EMA or frozen; stop-grad)
    ẑ  ← h(zc, a, horizon)                           # predicted target latent
    L  ← D(ẑ, zt) + λ_var·var_term(zc) + λ_cov·cov_term(zc)   # VICReg terms if g is trainable
    step(optimizer, L)
    g ← ema(g, f)                                   # if using EMA target (skip if frozen)
```

Algorithm 3 — Latent-space planning (the harness's planner).

```
zc ← f(current_business_context)
for each candidate action sequence A = (a_1..a_k):     # sampled or CEM-optimized
    z ← zc; cost ← 0
```

```
    for a in A:
        z ← h(z, a, horizon)                        # roll forward in latent space
        cost += value_head.cost(z)                  # objective: -operating profit, etc.
    score(A) ← cost
return argmin_A score(A)                            # MPC: propose best action, re-plan next step
```

Algorithm 4 — Per-business grounding.

```
ctx ← recent_business_views (censored, lagged)      # observation contract
if enough_history: predictor ← LoRA_finetune(h, ctx)  # encoder usually frozen
demand_dist ← demand_head(f(ctx))                   # de-censored by §3
plan        ← Algorithm 3 with f(ctx) as start
return {demand_dist, plan}                          # consumed by the harness, under approval tier
```

8. Evaluation

The protocol parallels the generative pipeline's, adapted to a model whose output is a representation rather than a likelihood. Evaluation of a joint-embedding model is necessarily *indirect* — through probes and planning, not a single held-out likelihood — which is a real cost of this route and stated as such.

8.1 Splits. Held-out seeds (in-distribution), held-out θ-regions (generalization across the industry — the decisive sim test), and real-business shadow logs (sim-to-real). Splits are by θ-region, not time alone.

8.2 Metrics.

Aspect	Metric
Representation quality	linear-probe accuracy/CRPS of demand and state recovered from the context latent
De-censoring	true-demand recovery error on stockout-censored items, vs the generative pipeline and classical estimators
Planning	regret of latent-MPC actions vs an oracle policy in sim; realized operating profit
Transfer	drop in the above from held-out θ-regions and from sim to real
Collapse diagnostics	embedding variance and covariance (a silent failure if unmonitored)

8.3 The decisive comparisons. Two. First, against baselines, as in the position paper: does the grounded model beat classical per-business methods and a generic model without a world model, most on censored items? Second, and specific to this paper: **head-to-head against the generative pipeline** on identical corpora and splits — does predicting in latent space transfer better (held-out θ), de-censor better, or plan more robustly over long horizons (where token-level rollout compounds error)? If it does none of these, the generative route is the right default and this paper's contribution is a negative result, which is still worth having.

8.4 Ablations. EMA vs VICReg vs frozen-encoder for collapse control; flat vs hierarchical (H-JEPA); paired-view privileged target vs same-view target (isolates the §3 contribution); action-conditioned vs unconditioned predictor.

9. When to prefer this over the generative pipeline

A decision guide, since the two papers propose competing routes.

Prefer the **generative** pipeline when…	Prefer the **joint-embedding** pipeline when…
You need explicit, calibrated next-event distributions as the primary product	The primary need is robust decisions and de-censored latent state
Clean likelihood-based evaluation matters for stakeholders	You can invest in probe-based evaluation and collapse monitoring
The team wants battle-tested, low-surprise training	The team can absorb research-grade SSL machinery
Surface detail (exact events) is itself the deliverable	Surface detail is mostly nuisance and long-horizon planning matters
Time-to-first-result dominates	Transfer quality and planning robustness dominate

The routes are not exclusive. A pragmatic program runs the generative pipeline as the default and the joint-embedding pipeline as the research bet, on the same corpus, and lets §8.3 decide — possibly ending with a hybrid: a joint-embedding backbone for state and planning, with a generative head where explicit event distributions are required.

10. Risks and failure modes

- **Representation collapse.** The defining risk of this route and absent from the generative one. Mitigation: EMA target, stop-gradient, VICReg variance-covariance terms, or a frozen encoder (DINO-WM); monitor embedding variance as a first-class metric, because collapse is silent and zeroes the loss.
- **The readout gap.** Everything actionable is a probe head on the latent, and a head can fail to recover a decision-relevant quantity the latent quietly dropped. Mitigation: validate that probes recover every quantity the harness consumes; if one is unrecoverable, the encoder discarded signal it should have kept.
- **Privileged-target faithfulness.** The §3 advantage is only as good as the simulation's truth view. If the generator's "true demand" is wrong, the model learns to predict a wrong latent confidently. Mitigation: the truth view is mechanistic, not modelled, and is validated against the same calibration anchors as the generator.
- **Indirect evaluation.** Without a likelihood, "is the model good?" is answered through probes and planning, which is slower and noisier to iterate on. Mitigation: fix probe protocols early; treat §8.3's head-to-head as the gate.
- **Maturity.** Joint-embedding world models are demonstrated in vision and robotics, not on heterogeneous business event streams. This is the novelty and the risk in one sentence. Mitigation: start from the frozen-encoder configuration, the most robust known recipe, before attempting end-to-end training.
- **Reality gap and parameter bias.** Inherited from the generator and unchanged from the companion pipeline; the same mitigations (θ coverage, shadow-mode evaluation, human oversight at high blast radius) apply.

11. Conclusion

The generative pipeline asks the model to predict everything that happens; this one asks it to predict only what the future *means* and to ignore the rest. The case for the second is unusually strong in this domain for two reasons that do not hold in general: the nuisance is nameable, so a mechanistic generator can withhold the detail that should be ignored; and the quantity the business most needs — true demand behind censored sales — is a latent, which is precisely what a joint-embedding model is built to predict. The simulation's ability to emit paired censored and uncensored views turns de-censoring into representation learning, a training signal no real dataset can offer.

It is also the riskier route — collapse-prone, evaluated indirectly, unproven on this kind of data. The honest plan runs it beside the generative default on a shared corpus and lets the head-to-head in §8.3 decide, with a hybrid as the likely endpoint. Either way the question is concrete and measurable: does predicting in latent space, trained on views only a simulation can pair, transfer and plan better than predicting the next event? The corpus to answer it is the same one either pipeline already builds.

References

- Assran, M., et al. (2023). Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture (I-JEPA). *CVPR 2023.* arXiv:2301.08243. https://arxiv.org/abs/2301.08243
- Bardes, A., Ponce, J., & LeCun, Y. (2022). VICReg: Variance-Invariance-Covariance Regularization for Self-Supervised Learning. *ICLR 2022.* arXiv:2105.04906. https://arxiv.org/abs/2105.04906
- Bardes, A., et al. (2024). Revisiting Feature Prediction for Learning Visual Representations from Video (V-JEPA). arXiv:2404.08471. https://arxiv.org/abs/2404.08471
- Grill, J.-B., et al. (2020). Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning (BYOL). *NeurIPS 2020.* arXiv:2006.07733. https://arxiv.org/abs/2006.07733
- Hafner, D., et al. (2025). Mastering diverse control tasks through world models (DreamerV3). *Nature.* https://www.nature.com/articles/s41586-025-08744-2
- Hansen, N., Su, H., & Wang, X. (2024). TD-MPC2: Scalable, Robust World Models for Continuous Control. *ICLR 2024.* arXiv:2310.16828. https://arxiv.org/abs/2310.16828
- LeCun, Y. (2022). A Path Towards Autonomous Machine Intelligence. *OpenReview.* https://openreview.net/forum?id=BZ5a1r-kVsf
- Oquab, M., et al. (2023). DINOv2: Learning Robust Visual Features without Supervision. arXiv:2304.07193. https://arxiv.org/abs/2304.07193
- V-JEPA 2 team (2025). V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning. arXiv:2506.09985. https://arxiv.org/abs/2506.09985
- Zhou, G., Pan, H., LeCun, Y., & Pinto, L. (2024). DINO-WM: World Models on Pre-trained Visual Features enable Zero-shot Planning. arXiv:2411.04983. https://arxiv.org/abs/2411.04983
- FreshRetailNet-50K (2025). A Stockout-Annotated Censored Demand Dataset for Latent Demand Recovery and Forecasting in Fresh Retail. arXiv:2505.16319. https://arxiv.org/abs/2505.16319
- Gneiting, T., & Raftery, A. E. (2007). Strictly Proper Scoring Rules, Prediction, and Estimation. *JASA*, 102(477), 359–378. https://doi.org/10.1198/016214506000001437
- Tobin, J., et al. (2017). Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World. *IROS 2017.* arXiv:1703.06907.

https://arxiv.org/abs/1703.06907
- Villalobos, P., et al. (2022, rev. 2024). Will we run out of data? Limits of LLM scaling based on human-generated data. *Epoch AI.* arXiv:2211.04325. https://arxiv.org/abs/2211.04325
- Shumailov, I., et al. (2024). AI models collapse when trained on recursively generated data. *Nature*, 631, 755–759. https://www.nature.com/articles/s41586-024-07566-y